

Vision, Architecture and Plan

AI-native spatial computing environment for Sorabbyngo OS

Pre-alpha design document · October 2026

Sorabbyngo Company · The Rhythm of Digital Life

1 Overview

Vanta is the spatial and interactive environment of Sorabbyngo OS. It is not designed as a traditional Linux desktop. It is the human-facing layer through which people see, touch and direct the **intelligent objects, spaces, agents and capabilities** that the Sorabbyngo platform provides.

Vanta combines a Wayland compositor with an AI-native interaction model and the **ObjectSpace** runtime. Users work with objects and spaces instead of applications and files, and every existing Linux application keeps working.

> **In one line:** ObjectSpace defines the world. Vanta is how the user experiences and manipulates that world.

> **Status: pre-alpha, in design.** Nothing in this document is implemented yet. It is the plan, and it will change as we build.

2 Vision

Traditional desktop computing is organized around applications: the user opens an application, which opens a file, inside an operating system. Vanta changes the model. The user states what they want or manipulates objects directly, and applications become implementation details.

A document is not merely a file. A workspace is not merely a collection of windows. An application is not something the user needs to think about. Vanta is the environment in which these ideas become directly manipulable.

Two paths lead into the system, and Vanta is where both begin and end:

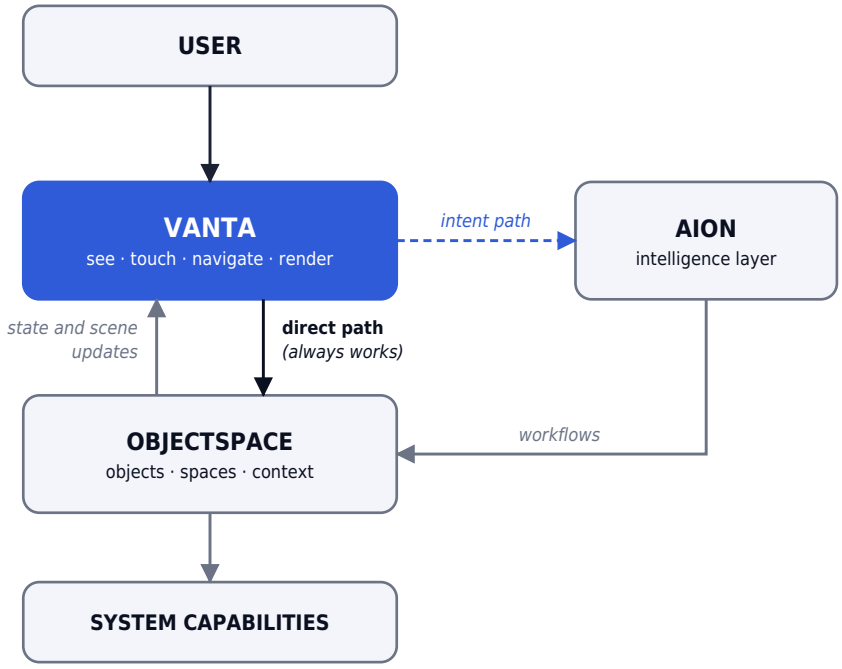


Figure 1. The direct path (solid) always works. The intent path (dashed) goes through AION and is optional. In both cases Vanta captures the input and presents the result, and ObjectSpace holds the truth.

- **Direct path:** User → Vanta → ObjectSpace → Vanta renders the result.
- **Intent path:** User → Vanta → AION → ObjectSpace → Vanta renders the result.

3 Principles

The first group describes what Vanta is. The second group are engineering rules that make the vision reliable. Every feature has to respect both.

Product principles

Principle	In practice
Intent-first, never intent-only	Users say what they want to achieve. Intent is the default way to work, and the direct path (mouse, keyboard, touch) always stays available.
Objects instead of files	Information is an intelligent, contextual object provided by ObjectSpace, not just a file on disk.
Spaces instead of windows	Work lives in persistent spaces and contexts rather than loose collections of windows.
AI-native	AI is part of the environment itself, not only an application inside it.
Spatial interaction	Objects, spaces and capabilities can be shown and manipulated spatially. This arrives in later phases.
Context-aware	Relationships between objects, tasks, spaces and sessions are preserved.
Local-first	Core functionality stays useful locally and does not need a permanent cloud connection.
Privacy-first	User data, context and interactions remain under the user's control.
Linux-compatible	Existing Linux software keeps working while the new interaction model grows above it.

Engineering rules

Rule	What it means
The direct path always works	Every action can be done without AI. Unplug the AI and the desktop must still feel complete.
The compositor never waits on a model	Windows appear instantly. AI-informed placement arrives afterward as an animation, never before.
One door for every action	Shortcuts, the GUI and AION all send the same typed actions to the same API. The AI has no private powers.
Predictable and undoable	Every AI action is visible and can be undone with one gesture. Destructive actions ask first.
No config files required	Every setting has a GUI and can be changed by asking. Files exist internally, but nobody has to open them.
Light by default	Small footprint and quick start. Spatial features cost nothing when unused, and effects never quietly drain the battery.
Simple for non-technical users	Familiar behavior first: windows, a launcher, a panel, settings. Power features are discoverable, never required.

4 Architecture

Vanta sits between the user and ObjectSpace. It depends on ObjectSpace for what exists and what it means, and it owns how all of that is shown and touched.

4.1 The stack

YOU	Keyboard, mouse, touch, voice, text. Gestures later.
VANTA	Compositor, window system, shell, settings, intent client, rendering and animation. How the world is seen and touched.
AION	The intelligence layer. Interprets intent and plans workflows.
OBJECTSPACE	Objects, spaces, relationships, context, memory, sessions, permissions, synchronization.
PLATFORM SERVICES	Byngox · Byngonet · HAL · CML · Sentinel
CORE AND KERNEL	Sorabbyngo Core and the Sorabbyngo OS kernel.
LINUX AND HARDWARE	Linux compatibility layer, drivers, hardware.

Figure 2. Vanta in the Sorabbyngo OS stack, following the ObjectSpace documentation.

4.2 Who owns what

Concern	ObjectSpace	Vanta
Objects	Defines what an object is: identity, state, permissions, history.	Shows objects and lets the user select, move and open them.
Spaces	Defines which objects belong together and keeps sessions persistent.	Lays a space out on screen: tiling, floating, transitions.
Context	Stores relationships and memory.	Displays context and uses it for placement hints.
Cross-device	Owns synchronization of objects, state and intent.	Presents spaces across devices and displays.
Rendering	Supplies the scene description.	Renders it: compositor, effects, animation.
Input	Receives the resulting actions.	Captures keyboard, mouse, touch, voice and gestures.

To agree: the ObjectSpace documentation also lists workspace management, rendering, gesture and projection runtimes. The split above is a proposal: ObjectSpace owns the meaning, Vanta owns the presentation. The exact contract must be settled with ObjectSpace and with the sorabbyngo-spatial work, which holds the canonical scene graph and the rendering engine.

4.3 Planned components

Component	Role
`vanta-comp`	Wayland compositor: window management, tiling and floating layouts, animations, rendering, XWayland.
`vanta-shell`	Panel, launcher, dock, notifications, lock screen, overview.
`vanta-settings`	Settings app and the daemon that stores and applies settings live.
`vanta-intent`	Vanta's client for AION: voice and text capture, on-screen feedback, confirmations and undo, plus a minimal local fallback for core commands. It is not a second AI brain.
`vanta-objects`	The bridge to ObjectSpace: projects objects and spaces into windows and scenes, and wraps legacy apps as objects.
`vanta-ipc`	The shared typed action API and message types used by every component.

Each component runs as its own process, so a failure in one cannot take down the others. The compositor in particular must survive a crash of anything above it.

5 Vanta Core

Vanta Core is the graphical foundation. Its responsibilities are:

- Wayland compositor, rendering, input and display management, including multi-monitor
- Window management with dynamic tiling, floating windows and workspaces
- Edge snapping, so a dragged window can fall into a tidy layout and float again when dragged back
- Scene management, animation, transitions and visual effects
- XWayland and full compatibility with existing Linux applications

The compositor is written in Rust on **Smithay**, the Rust library of building blocks for Wayland compositors (also used by COSMIC and Niri). Vanta can study the designs of Hyprland, KWin and others for tiling, spring and Bézier animation, overview and effects. Reusing their code depends on each project's license, and some are copyleft, so the license is settled early (see section 14).

To the compositor, a window is a surface with an application ID, a title and a size. Meaning such as "this belongs with your project" lives above it, in ObjectSpace and AION, and reaches the compositor as actions. That keeps the compositor small, fast and stable.

6 Spaces, objects and windows

The environment is organized around **spaces** and **objects**. A space can hold:

```
Project space
  ├── Documents
  ├── Source code
  ├── Research
  ├── Media
  ├── Agents
  └── Applications
```

Instead of treating these as unrelated windows, Vanta keeps their relationships through ObjectSpace, so a space is a persistent computing environment rather than a temporary pile of windows. ObjectSpace objects can represent documents, images, videos, projects, conversations, applications, devices, agents, workspaces, data and remote resources, each carrying context, relationships, permissions, history, state, location and cross-device identity.

> **Windows are one projection of an object.** The action API speaks in objects and spaces, not window IDs. A native object may appear as a card, a panel or a window. A legacy Linux application is wrapped as an object, so it can belong to a space while still appearing as an ordinary window.

7 Intent and AION

Vanta works with **AION**, the intelligence layer of Sorabbyngo OS. Instead of navigating application hierarchies, users express intent, for example:

```
"Prepare everything for my quarterly report."
```

The flow is: Vanta captures the request, AION interprets it and plans the workflow, ObjectSpace assembles the objects and the space, and Vanta presents the result with a smooth transition. The rules for this path are:

- AION proposes and ObjectSpace executes, always through the typed action API.
- Every action is shown to the user and can be undone. Destructive actions (closing, deleting) ask for confirmation.
- A model takes hundreds of milliseconds to seconds, while a frame takes about 16 ms. A new window appears instantly in a sensible default place, and an AI-suggested better position follows as an animation the user can undo.
- Placement must feel predictable. Vanta learns from corrections and does not move windows without a clear reason.
- If AION is slow, offline or wrong, nothing else is affected, because the direct path does not depend on it.

8 Actions and settings

Every operation is a typed **action**: move an object into a space, change a layout, focus, close, change a setting. Shortcuts, the GUI and AION all use the same actions, are logged the same way, and can be undone the same way.

Settings follow a schema and are stored by a settings daemon. Changes apply live, with no reload. The user changes a setting through the GUI (a slider for animation speed, drag and drop for layouts) or by asking. Files exist internally, but a user is never required to edit one.

9 Motion

Vanta treats motion as part of how the computer communicates, not as decoration. The environment is intended to support smooth space and workspace transitions, window morphing, spring and Bézier curves with per-event settings, contextual and focus transitions, dynamic layout changes, intelligent object placement and overview screens.

Hyprland already shows what is possible here, with its spring and Bézier curves and its tree of animation settings. Vanta builds its own interaction model on top of similar foundations. Effects that force a new frame on every screen refresh are used sparingly, because they cost battery.

10 Linux compatibility

Vanta is built for Sorabbyngo OS and stays compatible with the Linux ecosystem. Native objects and Linux applications both reach the screen through Wayland, and X11 applications run through XWayland. Users should not have to give up existing software to gain the benefits of an AI-native environment. Compatibility is also the reason the direct path matters: it is how every legacy application stays fully usable.

11 Spatial computing and cross-device spaces

Spatial interaction is Vanta's long-term differentiator and arrives in later phases. The interaction models planned include gesture-based object manipulation, spatial drag and drop, object projection, shared spaces, multi-device workspaces, external displays, projection surfaces and immersive environments.

For cross-device computing, the objective is not to synchronize files. It is to synchronize **objects, state, context and intent**, so an object can move between a PC, a phone, a tablet and a projector.

To avoid blocking this later, the first phases keep the action API and object model free of 2D-only assumptions, and the compositor is designed to consume the canonical scene graph and rendering engine from the sorabbyngo-spatial work once their contract is agreed.

12 Technology

- **Rust** for the compositor, shell, daemons and IPC. A compositor is long-running and must not crash, and Rust prevents the memory bugs that cause crashes.
- **Smithay** as the Wayland compositor foundation, with **Wayland** only as the session protocol and XWayland for X11 apps.
- **C and C++** at the edges, where they are already the best tools, such as local AI engines reached through bindings.
- **AION and ObjectSpace** are reached through stable interfaces, stubbed during early phases, so Vanta can be built and tested before they are finished.
- **Offline behavior** is inherited from AION and ObjectSpace. Vanta's own requirement is that direct manipulation and core commands work with no connection.

Planned repository layout:

```
assets/           logos and brand files
docs/             architecture, UX principles, shortcut map, this plan
crates/
  vanta-comp/     compositor
  vanta-shell/    panel, launcher, notifications
  vanta-settings/ settings app and daemon
  vanta-intent/   AION client, voice and text capture
  vanta-objects/  ObjectSpace bridge
  vanta-ipc/      shared action API
```

13 Roadmap

Each phase should leave something that runs. Spatial work comes last, but nothing earlier may block it.

Phase	Goal	Done when
0	Prototype the feel on Hyprland with a basic shell	Real users have tried the shortcuts, snapping and animation feel, and feedback is written down.
1	Compositor skeleton on Smithay, running nested in a window	A terminal and a browser render and take input inside it.
2	Window system: tiling, floating, snapping, workspaces, multi-monitor, animation engine	A full day of ordinary work is possible without leaving it.
3	Shell: panel, launcher, notifications, lock screen	A new user can start apps, switch and lock without help.
4	Action API, settings daemon and settings app	Every setting changes live from the GUI, with no config file.
5	Intent path: `vanta-intent` with AION, text first, then voice	Core commands work by text and voice, with undo and confirmation, and the desktop is fine when AION is off.
6	ObjectSpace integration: objects and spaces as windows, legacy apps wrapped as objects	A persistent space survives a restart with its objects and layout.
7	Installable Sorabbyngo OS session	Vanta appears in the display manager and installs by default on Sorabbyngo OS.

Phase	Goal	Done when
8	Spatial: gestures, projection, cross-device spaces, immersive surfaces	An object moves between two devices, and a gesture manipulates a space.

14 Risks and open questions

Risk	Mitigation
Scope: compositor, shell, settings, AI bridge and spatial work together is very large.	Phases that each run. Spatial last. A narrow v1 of Vanta Core plus the intent path.
Overlap with ObjectSpace on workspaces, rendering, gestures and projection.	Agree the ownership contract early (section 4.2), before Phase 4.
AI latency or wrong actions make the desktop feel unreliable.	AI stays out of the render path. Every action is undoable. The direct path always works.
Performance on modest hardware.	Light by default. Budget animations. Measure on low-end machines from Phase 1.
AION and ObjectSpace interfaces still moving.	Stable interface layer with stubs, so Vanta is not blocked.
License constraints when reusing other compositor code.	Decide Vanta's license early. Study designs freely. Check each source before copying code.

Open questions

- Which GUI toolkit for the shell and settings app?
- What is the exact contract with ObjectSpace and with the sorabbyngo-spatial scene graph and rendering engine?
- Default layout for first-time users: floating with easy snapping, or tiling?
- How does Vanta relate to Chameha, which the ObjectSpace documentation shows above AION?
- Which local models and hardware floor for the offline tier?
- Which license for Vanta?

Make computing feel less like operating applications and more like interacting with an intelligent digital environment.

Sorabbyngo · The Rhythm of Digital Life